

The **GWEB** System

A Literate Programming System for Go

(Version 0.1.0)

modeled on Knuth and Levy's **CWEB**

This is a guide to **GWEB** for readers who already know **CWEB**, or literate programming generally. It does not re-explain what literate programming is; it concentrates on how **GWEB** differs from **CWEB** and on the features and commands you will actually use. To a first approximation **GWEB** is “**CWEB** with C replaced by Go,” so most of your **CWEB** knowledge transfers intact — the interesting parts are where Go, and `gofmt`, change the picture.

GWEB at a glance

GWEB mirrors CWEB tool for tool:

- `gtangle` $\approx$ `ctangle` — extracts the compilable Go source;
- `gweave` $\approx$ `cweave` — typesets the documentation;
- `gwebmac.tex` $\approx$ `cwebmac.tex` — the $\text{\TeX}$ macros the woven output uses.

A web is a single file, conventionally `foo.w`. `gtangle foo` writes `foo.go` (plus any extra files); `gweave foo` writes `foo.tex`, which `pdfTeX` turns into `foo.pdf`. As in CWEB, the `.w` may be left off the command line.

What carries over from CWEB

If you know CWEB you already know most of GWEB. A web is a sequence of numbered *sections*, each with up to three parts in order — a $\text{\TeX}$ commentary part, a definition part (`@d/@f/@s`), and a code part — preceded by *limbo*. The control codes keep their familiar meanings:

- `@*`, `@*1`, `@*2`, ... start starred (chapter) sections; `@` starts an ordinary section.
- `@c` starts unnamed code; `@<Name@>=` defines a refinement and `@<Name@>` references one. A trailing ... abbreviates a name to its unique prefix.
- `@(file@>=` sends the following code to another output file; `@i` includes another web.
- `@f/@s` format an identifier; `@^text@>`, `@.text@>`, `@:text@>` add index entries; `@!` forces an index definition.
- `@=...@>` is verbatim, `@t...@>` is $\text{\TeX}$ text, `@&` pastes tokens, `@@` is a literal `@`, and `@%` and `@q...@>` are comments.
- `@`, `@/` `@|` `@#` fine-tune the woven layout.
- change files (`@x/@y/@z`) and inline code `|...|` in commentary work exactly as in CWEB.

Two small conveniences: `@p` is accepted as a synonym for `@c` (as in CWEB), and you never write `\input gwebmac` — `gweave` emits it for you, and removes a stray copy if you include one.

What makes GWEB different

The differences all follow from typesetting Go instead of C.

No preprocessor. Go has no macros, so `@d` has no analogue. It is accepted for portability but ignored by both tools; use Go `const` and `func` instead. The formatting directives `@f` and `@s` remain useful — for example `@f Stack int` sets the user type `Stack` in bold, the way predeclared types are set.

Pretty-printing mirrors gofmt, not a grammar. This is the central design choice, and the largest departure from `cweave`. `cweave` parses C and re-derives every space and break from the grammar. `gweave` does not parse Go at all: it reproduces the spacing that `gofmt` has already imposed. Because `gofmt`-formatted Go encodes the grammar in its spacing, this gets the hard cases right for free — the pointer `*T` versus the product `a * b`, the slice `[]T` versus the index `a[i]`, precedence spacing such as `a*b + c` — with no parser to maintain and no trouble with generics or future syntax. The practical rule is simply: *write the code in your sections the way gofmt would*.

A consequence is that CWEB's prettyprinter hints `@+`, `@[...@]`, and `@;` have nothing to do, and are accepted and ignored. The explicit layout codes `@`, `@/` `@|` `@#` still work for the occasional manual touch-up.

gtangle runs gofmt. The tangled output is passed through `gofmt`, so a correct web always yields canonically formatted Go. If the assembled text is not valid Go, `gtangle` keeps the unformatted output and prints a warning instead of failing.

User types are set bold automatically. `gweave` scans for type declarations — both `type Name ...` and the `type ( ... )` block form — and sets each declared type bold wherever it appears, the way `cweave` treats names introduced by a `typedef`. You can still classify a name yourself with `@f/@s`.

GWEB is written in itself. Like CWEB, GWEB is a literate program: its tools are tangled from the webs in `lit/`. The committed Go is byte-for-byte reproducible from those sources — a fixpoint the test suite checks on every change — so the program you read and the program that runs cannot drift apart.

The woven document

gweave produces the same furniture as **cweave**: run-in bold section headings (with a page break before each major starred section), a two-column index, a *Names of the Sections* list, a contents page, and section-number running heads. A whole section — its commentary and its code — is kept together on one page when it fits, and a code part is never split across a page boundary unless it is itself taller than a page.

How code is set. Reserved words and user-defined types are bold, other identifiers italic, string and rune literals typewriter (in `cmtex10`, as in CWEB), numbers and comments roman. A few Go-specific touches: the shift operators `<<` and `>>` are set as the tight symbols `<<<` and `>>>`; a thin space follows a function name or a keyword before `(`, so `f (x)` does not jam together; and a comment's `//` is kerned a little tighter. $\TeX$ mathematics written between dollar signs inside a comment is typeset as mathematics.

Index and formatting codes. The hand-index codes `@^text@` (roman), `@.text@` (typewriter), and `@:text@` (formatted by `\9`) work as in CWEB, as do `@!` to force a definition entry and `@f/@s` to reclassify a name. An identifier is marked as “defined” (its index number underlined) when it follows `func`, `var`, `const`, or `type`, or sits just left of `:=`; this is a heuristic, not a type analysis, and `@!` overrides it.

Hyperlinks and bookmarks. Section numbers shown as references, in the index, in cross-reference notes, and on the contents page become links, and the starred sections form a PDF outline. Two back ends produce them, exactly as in CWEB:

- with `pdftex` in PDF mode, the engine's own primitives;
- through DVI, by requesting the `\special` back end with `\let\pdf+` and converting the result with `dvipdfmx`.

A plain DVI run with no `\pdf` simply omits the links and outline.

Helper macros for commentary. `gwebmac.tex` carries the `cwebmac` niceties for prose: `\.{...}` sets typewriter text (with `\`, `\{`, ... for literal specials), `\{id}` sets an italic identifier, and the language tags `\CEE/`, `\GO/`, `\UNIX/`, `\TEX/` are available. `\,` is a thin space in math or text. The GWEB idiom for inline code, though, remains simply `|...|`.

Customizing the page. Redefine these in limbo (after the auto-emitted `\input gwebmac`):

- `\def\title{...}` — the running-head and contents title (it defaults to the upper-cased file name, and is shown all-caps in the header);
- `\pagewidth`, `\pageheight` — the page size;
- `\def\contentspagenumber{n}` together with `\pageno` — to renumber for binding into a larger document;
- `\Gcodepenalty` (how strongly a code part resists a page break) and `\Gcommentkern` (the `//` tightening) are dimensions you may tune.

Running the programs

```
gtangle [-o dir] [-line] file[.w] [change[.ch]]
gweave  [-o dir] file[.w] [change[.ch]]
```

The `.w` extension — and a change file's `.ch` — may be omitted. `-o` chooses the output directory. `-version` prints the version and exits. As each tool runs it prints a one-line banner and then a `*n` progress marker for every starred section, as CWEB's tools do.

`gtangle -line` is the Go counterpart of CWEB's `#line`: it emits `//line` directives in the tangled Go so the compiler, `go vet`, and panic traces report errors at `.w` positions rather than `.go` ones. It is off by default, leaving ordinary output (and the self-hosting fixpoint) unchanged.

To make a PDF, either go straight through `pdftex`, or through DVI and `dvipdfmx` for the same hyperlinks:

```
pdftex foo.tex           # native PDF, with links
```

```
tex "\let\pdf+ \input foo.tex"      # or via DVI...
dvipdfmx foo.dvi                  # ...then to PDF, with links
```

Make sure $\text{\TeX}$ can find `gwebmac.tex` (the installer below puts it on the search path; otherwise set `TEXINPUTS`).

Both tools report, each with a `file:line` location, an unterminated control code, a reference to an undefined section, an ambiguous `...` abbreviation, or a section defined but never used. These are warnings; `gtangle` stops only if it must actually expand a reference that was never defined.

Installing GWEB

For the commands alone, the Go way:

```
go install github.com/sjnam/gweb/cmd/gtangle@latest
go install github.com/sjnam/gweb/cmd/gweave@latest
```

For the full installation — the commands, `gwebmac.tex` placed in your $\text{\TeX}$ tree so `pdftex foo.tex` just works, and the man pages — run `./install.sh` from a checkout (`./install.sh --uninstall` removes it). The repository also includes a VS Code syntax highlighter for `.w` files under `editors/vscode`, and this manual is built with `make manual`.

The bundled examples

The `examples/` directory holds five worked webs, from simple to rich:

- `wc.w` — a word counter; shows `@f` and a change file (`wc.ch`);
- `pmap.w` — a generic concurrent map (goroutines, channels, a closure);
- `seq.w` — composable lazy iterators (generics, range-over-func);
- `floyd.w` — Floyd’s partition problem, after Knuth’s essay *Are Toy Problems Useful?*;
- `sham.w` — a port of Knuth’s Stanford GraphBase demonstration program `sham`, built on the `go-sgb` module.

GWEB also documents itself: `make selfdoc` weaves the system’s own `lit/*.w` sources into a single PDF.